14

Лекция 8–9 декабря

Схемы и верификация

Константин Чухарев

Дискретная математика — Осень 2026

Схемы и верификация

> *Что я не могу создать, я не понимаю.*
>
> ***— Ричард Фейнман***

Вентили

Вентиль — физический исполнитель булевой операции. На входах у него два уровня напряжения, на выходе один.

Вентиль	Функция	Смысл
NOT	$\overline{x}$	инвертирует
AND	$x \wedge y$	оба входа — 1
OR	$x \vee y$	хотя бы один — 1
XOR	$x \oplus y$	входы различны
NAND	$\overline{x \wedge y}$	отрицание AND
NOR	$\overline{x \vee y}$	отрицание OR

Пары читаются вопросами. AND спрашивает «оба — единицы?», OR — «хотя бы один?», XOR — «различны?», а NAND и NOR отрицают ответы AND и OR.

Таблицы истинности вентилей

Формула и таблица — два равноправных языка для одного и того же вентиля.

x	y	$x \wedge y$	$x \vee y$	$x \oplus y$	$\overline{x \wedge y}$	$\overline{x \vee y}$
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

NOT — единственный одновходовый вентиль набора, поэтому $\overline{0} = 1$ и $\overline{1} = 0$.

OR и XOR совпадают в трёх строках из четырёх и расходятся только при $x = y = 1$.

Русское «или» бывает исключающим («чай или кофе»), а вентиль OR — никогда. При двух единицах он честно выдаёт 1.

Спецификация аварийного реле

Реле аварии в серверной стойке читает три сигнала. x — перегрев, y — вентилятор охлаждения, z — кнопка теста. Таблица выписывает пять режимов срабатывания.

x	y	z	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Проверим строку $(1, 1, 0)$ — стойка греется, вентилятор работает, реле молчит.

Минтермы и СДНФ

Формулы у реле нет — только таблица.

Формулу собираем прямо по строкам таблицы. Кнопка теста z звонит в любом режиме, а без теста реле срабатывает только от перегрева при выключенном вентиляторе.

Минтерм — конъюнкция всех переменных, истинная ровно на одной строке. На месте единицы переменная входит в него без отрицания, на месте нуля — с отрицанием.

Выпишем минтермы всех пяти единичных строк.

$$\overline{x}\overline{y}z, \quad \overline{x}yz, \quad x\overline{y}\overline{z}, \quad x\overline{y}z, \quad xyz.$$

Каждый минтерм — детектор своей строки. Он равен 1 только на ней и 0 на всех остальных.

Дизъюнкция минтермов — совершенная ДНФ (СДНФ), равная 1 ровно там, где $f = 1$.

Склеивания минтермов

Пара минтермов, отличающихся ровно в одной переменной, склеивается — переменная исчезает.

$$\overline{x}\overline{y}z \vee \overline{x}yz = \overline{x}z, \qquad x\overline{y}\overline{z} \vee x\overline{y}z = x\overline{y}.$$

Ещё два таких склеивания дают $xyz \vee x\overline{y}z = xz$, затем $\overline{x}z \vee xz = z$.

$$f = z \vee x\overline{y}.$$

Из пяти минтермов уцелело два терма, и формула снова совпала со словесной формулировкой требования.

Склеивание работает только на соседних строках — отличающихся в одной позиции.

Карта Карно — та же операция, разложенная на клетках.

Из формулы в схему

Пример (Схема для реле)

Формуле $f = z \vee x\overline{y}$ отвечает цепочка из трёх вентилей.

- NOT выдаёт $\overline{y}$ — уровень 1.
- AND выдаёт $x \wedge \overline{y}$ — уровень 2.
- OR выдаёт $z \vee (x \wedge \overline{y})$ — уровень 3.

Схема из СДНФ до склеиваний требует 3 NOT, 10 AND и 4 OR. Это 17 вентилей и глубина 6.

Минимизация сократила её до 3 вентилей и глубины 3 — размер упал почти в шесть раз.

Проверяем вход $(1, 0, 0)$ — $\overline{y} = 1$, $x \wedge \overline{y} = 1$, реле срабатывает при перегреве и выключенном вентиляторе.

Схема как граф

Соединения вентилей образуют ориентированный граф, и все свойства схемы читаются по нему.

Определение 1 (Логическая схема)

Схема — ориентированный ациклический граф. Истоки в нём — входы, внутренние узлы — вентили, стоки — выходы.

Выходы одних вентилей подаются на входы других — схема вычисляет композицию булевых функций.

Релейная схема — такой граф с истоками x, y, z, тремя вентилями и одним стоком f.

Ацикличность даёт комбинационную логику, где результат зависит только от текущих входов, а цикл сделал бы значение неопределённым, потому что выход ждал бы сам себя.

Размер и глубина

Две схемы для одной функции различаются ценой, и цену считают двумя числами.

Определение 2 (Размер и глубина)

Размер схемы — число вентилей, то есть площадь и стоимость.

Глубина — длина самого длинного пути от входа к выходу, то есть задержка.

У реле из примера размер 3 и глубина 3.

В модели задержек каждый вентиль переключается за фиксированное время τ, а провода считаем мгновенными. Задержка сигнала — глубина, умноженная на τ, а самый длинный путь называется **критическим**. Такт процессора не короче худшего комбинационного пути — тактовая частота ограничена глубиной.

Размер и глубина — базовый инженерный компромисс. Быстрая схема собирается из большего числа вентилей, компактная работает медленнее.

Схема для XOR

Пример

$x \oplus y = (x \vee y) \wedge \overline{x \wedge y}$.

Формула читается по смыслу. AND ловит «оба — единицы», NOT даёт «не оба», OR собирает «хотя бы один», финальный AND требует обоих.

Вентилей четыре (OR, AND, NOT, AND), так что размер равен 4. Самый длинный путь $x, y \to \mathrm{AND} \to \mathrm{NOT} \to \mathrm{AND}$ даёт глубину 3.

Подадим $x = y = 1$, тогда $\overline{x \wedge y} = 0$ и выход равен 0, а OR даёт 1.

Подадим $x = 1$ и $y = 0$, тогда запрет снят, OR даёт 1 и выход равен 1.

Задержку задаёт критический путь. Параллельные ветви считаются одновременно, поэтому сумма задержек всех вентилей завышает ответ.

Полусумматор

Сложение начинается с младшего разряда, где складываются два бита без входящего переноса.

Определение 3 (Полусумматор (*half adder*))

Полусумматором называют схему, которая складывает два бита A и B.

$$S = A \oplus B, \quad C = A \wedge B.$$

S — младший бит суммы по модулю 2, поэтому его даёт XOR.

C — старший, и он равен 1 только при $A = B = 1$. Лишь там сумма переливается через разряд, поэтому его даёт AND.

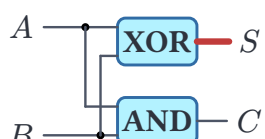


Таблица полусумматора

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Нижняя строка содержит всю механику переноса. Здесь $1 + 1 = 10_2$, младший бит уходит в S, старший — в C.

Приставка «полу-» честна. Столбик при сложении складывает три бита, а эта схема — только два.

Входящего переноса у полусумматора нет.

Полный сумматор

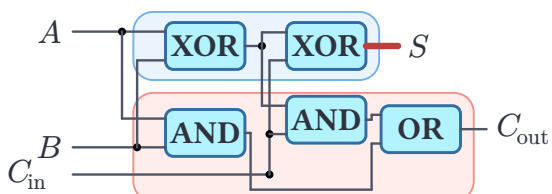
В следующем разряде к двум битам добавляется перенос из младшего.

Определение 4 (Полный сумматор (*full adder*))

Полным сумматором называют схему, которая складывает три бита — A, B и входящий перенос C_{in}.

$$S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)).$$

Подаются три бита, снимаются два. Бит суммы — чётность входов, а перенос уходит в старший разряд.



Чтение схемы по вентилям

Каждый провод назван формулой своего вентиля, и схема читается обратно в формулу.

Читаем схему переносов слева направо и снизу вверх.

- XOR и AND на входах дают $u = A \oplus B$ и $v = A \wedge B$ — уровень 1.
- Второй AND даёт $w = C_{\text{in}} \wedge u$ — уровень 2.
- OR на выходе даёт $C_{\text{out}} = v \vee w$ — уровень 3.

Подстановка снизу вверх разворачивает схему в одну формулу.

$$C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)).$$

Проверяем вход $(A, B, C_{\text{in}}) = (0, 1, 1)$ — $u = 1$, $v = 0$, $w = 1$, перенос равен 1, как и положено при $0 + 1 + 1 = 10_2$.

Схема и формула — два представления одной функции, и оба обратимы.

СДНФ переводит таблицу в схему, чтение по проводам — схему в формулу.

Каскад сумматоров

Разряд числа складывается одним полным сумматором, и перенос связывает их в цепочку.

Определение 5 (Ripple-carry)

Ripple-carry называют цепочку из n полных сумматоров, в которой перенос каждого разряда подаётся на вход следующего.

Размер $O(n)$, глубина $O(n)$.

Пример

Сложим $0111 + 0001$ и проследим перенос по разрядам.

В разряде 0 выходит $1 + 1 + 0 = 10_2$, то есть бит 0 и перенос 1. В разрядах 1 и 2 снова $1 + 0 + 1 = 10_2$, бит 0, а перенос идёт дальше. В разряде 3 выходит $0 + 0 + 1 = 01_2$, бит 1, перенос 0, волна гаснет, а сумма равна 1000.

Ripple-carry в вентилях

Пример (Точный счёт)

Полный сумматор — пять вентилей, а именно два XOR для суммы, два AND и OR для переноса.

Внутри одного разряда перенос проходит AND и OR, то есть две задержки.

Первый перенос готов за 3 задержки, каждый следующий — ещё за 2, финальный — за $2n + 1$.

Итог для n бит — размер $5n$ и глубина $2n + 1$.

4 бита — 20 вентилей и глубина 9, 64 бита — 320 вентилей и глубина 129.

Оценка $O(n)$ из определения развернулась в точные числа. Числа 5 и 2 — цена одного разряда.

Задача верификации

Схема спроектирована, затем оптимизирована — и нужно доказать, что поведение не изменилось.

Перебор всех входов невозможен, ведь у n входов 2^n наборов.

Решатель отвечает на вопрос, существует ли вход, на котором выходы двух схем различаются.

> Обе схемы переводят в КНФ. На каждый вентиль заводится переменная его выхода и дизъюнкты, задающие таблицу вентиля (преобразование Цейтина).
>
> Размер растёт линейно по числу вентилей.

Митер

Вопрос о различии двух схем сводится к одной схеме, у которой выход есть только при расхождении.

Определение 6 (Митер (*miter*))

Митером называют схему, в которой входы схем C_1 и C_2 соединены попарно, на каждую пару выходов стоит XOR, а выходы всех XOR собраны одним OR.

Митер выдаёт 1 ровно на тех входах, где схемы различаются. XOR срабатывает на расхождении пары выходов, OR собирает любые расхождения вместе.

Схемы эквивалентны тогда и только тогда, когда их митер невыполним, и вопрос об эквивалентности становится вопросом о выполнимости.

Митер в действии

Пример

Сравним $f_1(x, y) = x \vee y$ и $f_2(x, y) = x \oplus y$.

На входе $(1, 1)$ выходит $f_1 = 1$ и $f_2 = 0$. XOR выходов даёт 1, и митер срабатывает.

На входе $(1, 0)$ обе функции выдают 1, поэтому расхождения нет.

А схема $(x \vee y) \wedge \overline{x \wedge y}$ эквивалентна вентилю XOR. Их митер невыполним, и решатель доказал бы это без проверки наборов.

На двух входах хватило бы перебора, но у промышленных схем тысячи входов. 2^n наборов никто не перебирает, а UNSAT-ответ остаётся доказательством.

SAT — решатель предъявляет контрпример, вход, разводящий схемы.

UNSAT — доказательство эквивалентности на всех входах сразу.

Функциональная полнота

Релейное построение уже дало ответ, и НЕ, И и ИЛИ достаточно для любой таблицы.

Определение 7 (Функциональная полнота)

Набор вентилей называется **функционально полным**, если из него собирается схема для любой булевой функции.

Импликация в набор не входит, но собирается из него двумя вентилями, а эквивалентность — из двух импликаций.

$$x \to y = \overline{x} \vee y, \quad x \equiv y = (x \to y) \wedge (y \to x).$$

Производство спрашивает дальше, сколько типов вентилей выпускать, проверять и держать на складе.

Каждый выброшенный из базиса вентиль — минус один тип ячейки в библиотеке стандартных элементов.

Полнота НЕ, И и ИЛИ

Достаточность доказывается тем же построением, что и для реле.

Теорема 1 (Полнота построением)

Набор NOT, AND, OR функционален для любой функции от n переменных.

Доказательство (построением по таблице)

Для каждой строки с $f = 1$ выпишем минтерм, вентилями AND соберём конъюнкции, вентилями OR соединим их, а отрицания дадут вентили NOT.

Дизъюнкция минтермов равна 1 ровно на единичных строках, поэтому собранная схема вычисляет f.

Случай $k = 0$. Если единичных строк нет, функция равна нулю, и её даёт $x \wedge \overline{x}$.

Для $x \oplus y$ построение даёт два минтерма и пять вентилей, два NOT, два AND и один OR.

Шестнадцать функций двух входов

Функций от двух входов $2^4 = 16$, от n входов — 2^{2^n}. Слово из значений на наборах $00, 01, 10, 11$ задаёт функцию, а 0110 — это XOR.

Слово	Функция	Имя
0000	0	константа 0
0001	$x \wedge y$	конъюнкция
0010	$x\overline{y}$	запрет y
0011	x	проекция x
0100	$\overline{x}y$	запрет x
0101	y	проекция y
0110	$x \oplus y$	XOR
0111	$x \vee y$	дизъюнкция

Слово	Функция	Имя
1000	$\overline{x \vee y}$	$\downarrow$ — **полон**
1001	$x \equiv y$	эквивалентность
1010	$\overline{y}$	NOT y
1011	$y \to x$	обратная импликация
1100	$\overline{x}$	NOT x
1101	$x \to y$	импликация
1110	$\overline{x \wedge y}$	$\uparrow$ — **полон**
1111	1	константа 1

Полнота одного NAND

Из шестнадцати функций двух входов достаточно одной, и это NAND.

Теорема 2 (Штрих Шефера)

Один NAND выражает НЕ, И и ИЛИ.

$$\overline{x} = x \uparrow x, \quad x \wedge y = (x \uparrow y) \uparrow (x \uparrow y), \quad x \vee y = (x \uparrow x) \uparrow (y \uparrow y).$$

Доказательство

Проверим три тождества подстановкой $x \uparrow y = \overline{x \wedge y}$.

Дизъюнкция проверяется де Морганом, и равенство принимает вид $\overline{(x \uparrow x)} \uparrow \overline{(y \uparrow y)} = \overline{\overline{x}\,\overline{y}} = x \vee y$. Конъюнкция — двойное отрицание над NAND, поэтому $\overline{\overline{x \wedge y}} = x \wedge y$. При $x = y = 1$ выходит $1 \uparrow 1 = 0$, и $(1 \uparrow 1) \uparrow (1 \uparrow 1) = 0 \uparrow 0 = 1$.

Полнота наследуется, ведь каждая операция полной тройки выражена через NAND.

Почему полны только NAND и NOR?

Поодиночке из шестнадцати функций полны ровно две — NAND (второе имя — штрих Шеффера, *Sheffer stroke*) и NOR.

Остальные четырнадцать сохраняют хотя бы одно из пяти свойств, замкнутых относительно композиции, а именно тождественный ноль, тождественную единицу, монотонность, самодвойственность или линейность. NAND не сохраняет ни одного из пяти, поэтому ограничения на композицию у него нет.

Пример (Почему AND не даёт NOT?)

AND монотонен, и композиция монотонных функций монотонна. NOT убывает, ведь $0 < 1$, но $\overline{0} > \overline{1}$. Поэтому из одних AND отрицание не собрать.

В КМОП NAND дешевле AND, четыре транзистора против шести, потому что на выходе AND стоит инвертор, и библиотеки стандартных ячеек строятся вокруг NAND.

Код Грея

Помимо обычного счёта существует другой порядок тех же двоичных строк.

Определение 8 (Код Грея (*Gray code*))

Кодом Грея называют упорядочение всех 2^n двоичных строк длины n, в котором соседние строки — включая последнюю и первую — отличаются ровно в одном бите.

Пример

G_3: $000, 001, 011, 010, 110, 111, 101, 100$.

Каждая соседняя пара — один бит разницы.

Проверим стык. Последняя строка 100 и первая 000 различаются старшим битом, поэтому список замыкается в цикл.

Зачем код Грея?

При переключении нескольких битов сразу возникают ложные промежуточные состояния.

Двоичный счётчик на переходе $0111 \to 1000$ меняет все четыре бита, и энкодер успевает прочитать 0101 или 1111 — значения, которых не было никогда.

Код Грея меняет один бит за шаг — энкодеры угла и позиции читаются безошибочно.

Код Грея уже работал в карте Карно, где соседние клетки — соседние строки.

Построение кода Грея

Код длины $n+1$ собирается из кода длины n.

Определение 9 (Рекурсивное построение)

Рекурсивное построение кода Грея начинается с $G_1 = [0, 1]$.

$$G_{n+1} = 0G_n, 1G_n^R,$$

где $0G_n$ — слова G_n с нулём слева, а $1G_n^R$ — те же слова в обратном порядке с единицей слева.

Пример (От G_1 к G_3)

Из $G_1 = [0, 1]$ выходят половины $0G_1 = [00, 01]$ и $1G_1^R = [11, 10]$, а вместе $G_2 = 00, 01, 11, 10$. Ещё один шаг приписывает ноль к G_2 и единицу к обращённой G_2^R, и получается список из восьми строк.

Почему рекурсия даёт код Грея?

Определение задаёт список, но не доказывает, что он обладает нужным свойством.

Теорема 3

Список $0G_n, 1G_n^R$ перечисляет все строки длины $n+1$ и меняет ровно один бит на каждом шаге, включая замыкание.

Доказательство

Докажем индукцией по n.

База. Пара $[0, 1]$ отличается в единственном бите.

Шаг. Пусть G_n перечисляет все строки длины n без повторов и меняет один бит на каждом шаге. Ноль слева и обращение сохраняют соседство, поэтому половины наследуют свойство, различаются старшим битом и дают все 2^{n+1} строк по разу, а стык и замыкание — только старшим битом.

Перевод между кодами

Обе нумерации задают одни и те же 2^n строк, поэтому между ними есть прямой перевод.

Теорема 4 (Перевод через XOR)

Из двоичного кода в код Грея ведёт $g_i = b_i \oplus b_{i+1}$, старший бит сохраняется.

Обратно работает накопительный $\oplus$ от старшего бита, $b_i = g_i \oplus b_{i+1}$.

Пример

Для $b = 1011$ выходит $g = 1110$, старший бит тот же, дальше $\oplus$ соседей.

Обратно по $g = 1110$ получаем $b_3 = 1$, $b_2 = 1 \oplus 1 = 0$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 0 \oplus 1 = 1$. Мы вернулись к исходному 1011.

Прямое преобразование — один слой XOR, обратное — цепочка глубины $O(n)$.

Верхняя оценка цены СДНФ

СДНФ даёт схему для любой таблицы, и её размер оценивается сверху.

Теорема 5 (Схема из таблицы)

Функция от n переменных, равная единице на k наборах, вычисляется схемой размера не более

$$n + (n-1)k + (k-1).$$

Доказательство

Считаем вентили СДНФ-схемы. Отрицаний не больше n, по одному на переменную с веером на все минтермы, конъюнкций $(n-1)k$, дизъюнкций $k-1$.

При максимальном $k = 2^n$ оценка даёт $n \cdot 2^n + n - 1$ — экспоненту от числа входов. Реле — точно этот случай с $n = 3$ и $k = 5$, и оценка даёт $3 + 2 \cdot 5 + 4 = 17$, те же семнадцать вентилей, что вышли прямым счётом.

Чётность в двух базисах

Пример (Чётность n бит)

Функция равна 1, когда единиц во входе чётное число.

Единичных наборов половина, а именно 2^{n-1}, и любые два из них различаются минимум в двух битах. Склеивать нечего, поэтому минимальная ДНФ чётности — её СДНФ.

Порядок размера — $n \cdot 2^{n-1}$ вентилей против цепочки из $n - 1$ XOR. При $n = 32$ это $6.9 \cdot 10^{10}$ против тридцати одного.

Одна функция — экспонента в базисе НЕ, И, ИЛИ и линейная схема в базисе с XOR.

Цена функции зависит и от самой функции, и от того, какие вентили объявлены базисом.

Семейства схем

Одна схема имеет фиксированное число входов, поэтому схема для 4-битного сложения не сложит 8-битные числа.

Определение 10 (Семейство схем)

Семейство схем — последовательность $C_1, C_2, \ldots$, где C_n имеет n входов.

Семейство **равномерно**, если существует алгоритм, строящий C_n по n.

Для чётности C_n — цепочка из $n - 1$ XOR-вентилей.

Равномерные полиномиальные семейства — схемный взгляд на эффективные вычисления.

Функция переноса как большинство

Перенос равен 1 тогда и только тогда, когда хотя бы два из трёх входов равны 1, то есть перенос — функция большинства трёх входов.

Совпадение с формулой схемы проверяется раскрытием $\oplus$ и поглощением.

$$(A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)) = (A \wedge B) \vee (C_{\text{in}} \wedge A) \vee (C_{\text{in}} \wedge B).$$

Справа стоит большинство, выписанное напрямую, тремя конъюнкциями по одной на каждую пару входов.

Проверим на битах. При $(0, 1, 1)$ единственная полная пара даёт $C_{\text{in}} \wedge B = 1$, и перенос равен 1, а при $(0, 1, 0)$ не собрана ни одна пара и перенос равен 0.

Формулы полусумматора и полного сумматора — минимизированные ДНФ.

Из минимизированных формул собирается арифметика процессора.

Работа и глубина

За сложением стоит общий торг двух ресурсов.

Работа — суммарное число вентильных операций, у схемы это размер, то есть площадь и энергия.

Глубина — параллельное время, то есть сколько уровней логики обязан пройти сигнал.

Каскад для сложения даёт работу $5n$ при глубине $2n + 1$.

Опережающий перенос (*carry-lookahead*) считает переносы группами и снижает глубину до $O(\log n)$, а работа растёт.

Цепочка экономит площадь, дерево покупает время, и оба ресурса сразу не выиграть.

> Вычитание тоже каскад, ведь $A - B = A + \overline{B} + 1$ в дополнительном коде, а вся арифметика процессора — вариации одного сумматора.
>
> В программе тот же выбор. Параллелизм оплачивается дублированием работы.

Глубина нетлиста

Нетлист — схема в тексте, то есть список вентилей в топологическом порядке.

```
use std::collections::HashMap;

let gates: [(&str, &[&str]); 3] = [
    ("g1", &["y"]),
    ("g2", &["x", "g1"]),
    ("g3", &["z", "g2"]),
];

let mut level = HashMap::from([("x", 0), ("y", 0), ("z", 0)]);
for (out, ins) in gates {
    let d = 1 + ins.iter().map(|w| level[*w]).max().unwrap();
    level.insert(out, d);
}
println!("{}", level["g3"]); // 3
```

Глубина нетлиста [2]

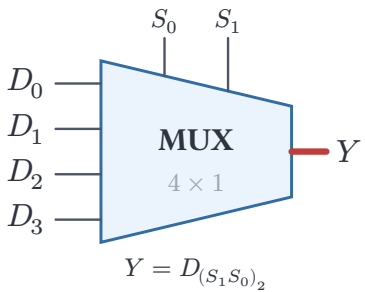
Один проход считает уровни всех проводов. Уровень вентиля — максимум уровней его входов плюс один.

Тот же проход делают симуляторы схем, а для SAT нетлист переводится в КНФ.

АЛУ

Арифметико-логическое устройство — вычислительное ядро процессора.

Однобитный срез — это блоки AND, OR, сложения и вычитания на общих входах A, B. Мультиплексор по адресу $S_1 S_0 = 10$ выводит на Y выбранный D_2.



Мультиплексор «из 2^k в 1» выдаёт один из 2^k входов по k адресным битам.

n-битное АЛУ — n таких срезов с цепочкой переносов.

Дизъюнкт как импликация

Митер свёл эквивалентность схем к вопросу о выполнимости КНФ.

Дизъюнкт из двух литералов равносилен паре импликаций.

$$l_1 \vee l_2 \quad \equiv \quad \overline{l_1} \to l_2, \quad \overline{l_2} \to l_1.$$

Вторая импликация — контрапозиция первой.

Потерять её — потерять половину рёбер будущего графа.

Тройку литералов так разложить нельзя — граница между лёгким и трудным случаями SAT.

Граф импликаций

Разложение в импликации превращает формулу в ориентированный граф, и её выполнимость читается по этому графу.

Определение 11 (Граф импликаций)

В графе импликаций вершины — все литералы, x и $\overline{x}$ для каждой переменной.

Рёбра — обе импликации каждого дизъюнкта.

Выполнимость превращается в вопрос о достижимости в графе.

Путь в графе — цепочка принуждений, где истинность начала тянет за собой истинность конца.

Поэтому конфликтом грозит именно цикл, собирающий пару $x, \overline{x}$.

Критерий 2-SAT

Выполнимость читается по компонентам сильной связности графа импликаций.

Теорема 6 (Критерий 2-SAT)

Формула 2-SAT невыполнима тогда и только тогда, когда некоторый литерал и его отрицание лежат в одной компоненте сильной связности графа импликаций.

Доказательство

Докажем критерий в обе стороны. Если x и $\overline{x}$ взаимно достижимы, пути дают $x \to \overline{x}$ и $\overline{x} \to x$, поэтому ни одно значение не годится.

Пусть теперь пары разнесены по компонентам, и литерал истинен, когда его компонента идёт позже компоненты отрицания. Если ребро $A \to B$ нарушено, то B идёт раньше $\overline{B}$, контрапозиция даёт $\overline{B} \leq \overline{A}$, а $\overline{A}$ идёт раньше A. Вместе с $A \leq B$ цепочка замыкается в $B < B$.

Построение выполняющего набора

Если пары разнесены по компонентам, модель читается из графа, а компоненты находит обход в глубину за $O(n + m)$.

1. Найти компоненты сильной связности и топологический порядок их конденсации.
2. Обрабатывать компоненты в обратном порядке — от стоков к истокам.
3. Переменная получает 1, если её компонента идёт позже компоненты отрицания, иначе 0.

Обратный порядок гарантирует совместимость. К моменту решения о литерале все зависимые от него литералы уже получили значения.

По пути в графе значение не убывает, ложь идёт перед истиной.

Поэтому каждая импликация оказывается истинной.

Пример графа импликаций

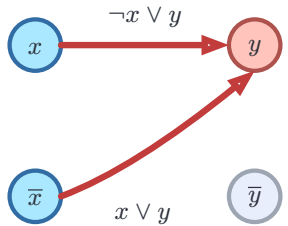


Рис. 1. Контрапозиции опущены.

Пример

Возьмём формулу $(x \vee y) \wedge (\overline{x} \vee y)$.

Если $x = 0$, первый дизъюнкт требует $y = 1$. Если $x = 1$, второй требует того же, поэтому y истинна при любом значении x.

Пары $x, \overline{x}$ и $y, \overline{y}$ лежат в разных компонентах, и критерий подтверждает выполнимость. Циклов в графе нет, поэтому каждая компонента — одна вершина.

Полный экземпляр 2-SAT

Пример (Три дизъюнкта, шесть рёбер)

$\varphi = (x \vee y) \wedge (\overline{x} \vee z) \wedge (\overline{y} \vee \overline{z})$.

Из дизъюнктов выходят рёбра $\overline{x} \to y$, $\overline{y} \to x$, $x \to z$, $\overline{z} \to \overline{x}$, $y \to \overline{z}$, $z \to \overline{y}$.

Циклы собираются в две компоненты, $\{\overline{x}, y, \overline{z}\}$ и $\{x, z, \overline{y}\}$. Ни одна из них не содержит пары $x, \overline{x}$.

Формула выполнима, и модель читается прямо из компонент, $x = 1$, $z = 1$, $y = 0$.

Подстановка даёт $(1 \vee 0) = 1$, $(0 \vee 1) = 1$, $(1 \vee 0) = 1$ — все три дизъюнкта истинны.

2-SAT против 3-SAT

2-SAT решается за линейное время.

3-SAT — SAT, где каждый дизъюнкт содержит ровно три литерала, — NP-полна.

Один лишний литерал — скачок сложности.

Причина структурная. Пара литералов распадается на импликации, а тройка — нет, и графа импликаций для 3-SAT не строится.

Вспомогательные переменные Цейтина сохраняют выполнимость, но формула после них всё равно содержит тройки, а не пары. Иначе 3-SAT сводилась бы к 2-SAT, и SAT решался бы за полиномиальное время.

Хорновские дизъюнкты

Ограничение на знак литерала даёт класс формул с линейным алгоритмом проверки.

Определение 12 (Хорновский дизъюнкт)

Дизъюнкт, в котором не больше одного положительного литерала, называется **хорновским** (*Horn clause*).

$$\overline{p_1} \vee \ldots \vee \overline{p_k} \vee q.$$

Эквивалентная запись — правило $p_1 \wedge \ldots \wedge p_k \to q$.

При $k = 0$ правило без посылок — факт q. Дизъюнкт без положительных литералов — цель.

Один дизъюнкт играет три роли. Правило с посылками читается «если, то», факт служит данными, цель служит запросом.

Примеры хорновских дизъюнктов

Пример

$(\overline{x} \vee \overline{y} \vee z)$ — правило $x \wedge y \to z$.

$(\overline{x} \vee y)$ — правило $x \to y$.

(x) — факт, а $(\overline{x} \vee \overline{y})$ — цель «не оба сразу».

Хорновские формулы — язык правил, и база знаний состоит из фактов и правил вывода.

На них стоит логическое программирование, а вывод типов в компиляторах — решение хорновской системы ограничений.

Хорн за линейное время

Распространение начинается с фактов. Правило с истинными посылками помечает свой вывод, пока очередь не опустеет, а пустая цель даёт пустой дизъюнкт.

Теорема 7

Выполнимость хорновской формулы решается за линейное время.

Доказательство

Докажем корректность и оценку времени.

Корректность. Каждая пометка вынуждена, поэтому непомеченные переменные можно объявить ложными, и тогда у каждого несработавшего правила есть ложная посылка.

Сложность. Дизъюнкт срабатывает один раз, и проверки ложатся на его литералы.

Наименьшая модель

Распространение даёт одну конкретную модель, и она наименьшая из всех.

Определение 13 (Наименьшая модель)

Модель, истинная ровно на принуждённых литералах, называется **наименьшей моделью**. У выполнимой хорновской формулы она одна.

Пример (Прогон)

$\varphi = (x) \wedge (\overline{x} \vee y) \wedge (\overline{z} \vee w)$.

Факт x принуждает y из правила $x \to y$, и распространение заканчивается. Переменные z и w никем не принуждаются, поэтому наименьшая модель $x = 1, y = 1, z = 0, w = 0$.

Добавим цель $(\overline{x} \vee \overline{y})$, и после $x = 1, y = 1$ она пустеет, так что ответ UNSAT.

Наименьшая модель объясняет, почему порядок правил в программе не влияет на ответ.

Единичное распространение

Подстановка найденных значений сокращает формулу, и дизъюнкты решаются сами.

Определение 14 (Единичное распространение (*unit propagation*))

Единичное распространение — правило, по которому, если в дизъюнкте все литералы, кроме одного, ложны, оставшийся полагают истинным. Такой дизъюнкт называется **единичным**.

Пример (Каскад без ветвлений)

$(x) \wedge (\overline{x} \vee y) \wedge (\overline{y} \vee z) \wedge \overline{z}$.

Единичный дизъюнкт (x) даёт $x = 1$. Дизъюнкт $(\overline{x} \vee y)$ становится единичным и даёт $y = 1$.

Затем $z = 1$ — и последний дизъюнкт ложен.

Формула невыполнима, противоречие найдено без единого ветвления.

Правила DPLL

DPLL спускается по формуле двумя правилами, и оба сохраняют выполнимость. Вычёркивается только заведомо невыполнимое.

Первое правило — единичное распространение. Второе правило — чистый литерал.

Определение 15 (Чистый литерал)

Литерал **чист**, если его отрицание не встречается в формуле.

Чистому литералу можно смело дать единицу. Он не опровергает ни один дизъюнкт, и его дизъюнкты вычёркиваются целиком.

Ветвление и конфликт в DPLL

Когда оба правила исчерпаны, а дизъюнкты остались, формула требует выбора — и поиск ветвится.

Определение 16 (Ветвление и конфликт)

- При **ветвлении** выбирается неприсвоенная переменная, и оба её значения опробуются по очереди.
- **Конфликт** — пустой дизъюнкт, поэтому ветвь закрывается, и поиск возвращается к последнему выбору.

Оба правила не теряют выполнимость. Единичный литерал принуждён, а чистому годится единица.

Поэтому исчерпание всех ветвей доказывает невыполнимость формулы.

Прогон DPLL

Пример (Формула из четырёх дизъюнктов)

$$\varphi = (x \vee y) \wedge (\overline{x} \vee y) \wedge (x \vee \overline{y}) \wedge (y \vee z).$$

Единичных дизъюнктов нет, поэтому решатель ветвится, и первым делом пробует $x = 0$.

Отступы в трассе показывают глубину ветвления. Номер в трассе — это переменная. Единица обозначает x, двойка — y.

```
ветвим 1 = False
  unit   2
  конфликт
ветвим 1 = True
  unit   2
  модель {1: True, 2: True}
```

Прогон DPLL [2]

Ветвь $x = 0$ даёт единичные (y) и $(\overline{y})$, а в ветви $x = 1$ все дизъюнкты выполнены.

Ядро DPLL

Ядро умещается в полтора десятка строк. Литерал — целое со знаком, где 1 — это x, а -1 — это $\overline{x}$.

```
use std::collections::HashMap;

fn step(clauses: &[Vec<i32>], lit: i32) -> Vec<Vec<i32>> {
    clauses
        .iter()
        .filter(|c| !c.contains(&lit))
        .map(|c| c.iter().copied().filter(|&l| l != -lit).collect())
        .collect()
}

fn dpll(mut clauses: Vec<Vec<i32>>, mut model: HashMap<i32, bool>)
    -> Option<HashMap<i32, bool>>
{
```

Ядро DPLL [2]

```
    loop {
        let unit = clauses.iter().find(|c| c.len() == 1).map(|c| c[0]);
        let Some(unit) = unit else { break };
        model.insert(unit.abs(), unit > 0);
        clauses = step(&clauses, unit);
        if clauses.iter().any(|c| c.is_empty()) {
            return None;
        }
    }
```

Ветвление в коде DPLL

Остаток функции выбирает литерал и вызывает себя на формуле, укороченной этим выбором.

```
    if clauses.is_empty() {
        return Some(model);
    }

    let lit = clauses[0][0];
    for val in [false, true] {
        let mut next = model.clone();
        next.insert(lit.abs(), val);
        let reduced = step(&clauses, if val { lit } else { -lit });
        if let Some(m) = dpll(reduced, next) {
            return Some(m);
        }
    }
```

Ветвление в коде DPLL [2]

```
    None
}

dpll(vec![vec![1, 2], vec![-1, 2], vec![1, -2], vec![2, 3]], HashMap::new());
// (x y) (-x y) (x -y) (y z)
```

Трассировку можно обогатить чистыми литералами, а индустриальные решатели добавляют выученные дизъюнкты, и так из DPLL вырастает CDCL.

CDCL

Современные решатели — CDCL (*Conflict-Driven Clause Learning*), то есть DPLL плюс обучение на конфликтах.

- решение — эвристика VSIDS выбирает переменную с приоритетом частых участников недавних конфликтов.
- анализ конфликта — по истории присваиваний извлекается выученный дизъюнкт.
- нехронологический возврат и периодические перезапуски.

DPLL при конфликте просто отступает.

CDCL запоминает его причину — и не ходит туда второй раз.

Выученный дизъюнкт

Пример (Конфликт и резолюция)

Пусть решения дали $x_4 = 1$ и $x_5 = 1$.

Дизъюнкты $(\overline{x_4} \vee \overline{x_5} \vee x_6)$ и $(\overline{x_4} \vee \overline{x_5} \vee \overline{x_6})$ дают конфликт.

Первый требует $x_6 = 1$, после чего второй ложен.

Резолюция двух дизъюнктов по x_6 даёт **выученный дизъюнкт**.

$$\overline{x_4} \vee \overline{x_5}.$$

Комбинация $x_4 = x_5 = 1$ запрещена до конца поиска.

Выученные дизъюнкты — шаги резолюции.

Каждый отсекает целое подпространство перебора, а не одну ветвь.

Сертификат невыполнимости

Ответ UNSAT тоже предъявляет доказательство, и оно проверяется без доверия к решателю.

Простейший сертификат — пара дизъюнктов (x) и $(\overline{x})$. Их резолюция сразу даёт пустой дизъюнкт.

Цепочка (x), $(\overline{x} \vee y)$, $(\overline{y})$ требует двух шагов. Резолюция первых двух по x даёт (y), а резолюция (y) и $(\overline{y})$ даёт пустой дизъюнкт.

Решатель сохраняет цепочку резолюций, приведшую к пустому дизъюнкту.

Независимая проверяющая программа воспроизводит её за доли секунды.

Верить решателю не нужно, ведь доказательство проверяется механически.

Для митера доказательством эквивалентности схем служит невыполнимость формулы различий.

Кодирование раскраски графа

Раскраска — первая задача, которую можно закодировать дизъюнктами целиком.

Определение 17 (k-раскраска)

***k*-раскраска** графа $G = (V, E)$ — функция $V \to \{1, \ldots, k\}$, переводящая смежные вершины в разные цвета.

Переменная $x_{v,c} = 1$ означает, что вершина v получает цвет c.

Раскраску задают три группы ограничений.

- Каждая вершина получает цвет, а именно $\bigvee_{c=1}^{k} x_{v,c}$.
- Вершина получает не больше одного цвета, то есть $\overline{x_{v,c}} \vee \overline{x_{v,d}}$ для каждой пары $c < d$.
- Смежные вершины различаются, то есть $\overline{x_{u,c}} \vee \overline{x_{v,c}}$ для каждого ребра и цвета.

Всего $O(|V| \cdot k^2 + |E| \cdot k)$ дизъюнктов — полином от размера графа, и решатель получает экземпляр своей родной формы.

Треугольник в два цвета

Пример (Полный пример)

Граф — треугольник с попарно смежными вершинами a, b, c, а цветов у него $k = 2$.

У каждой вершины есть цвет, и это даёт три дизъюнкта.

$$\left(x_{a,1} \vee x_{a,2}\right) \wedge \left(x_{b,1} \vee x_{b,2}\right) \wedge \left(x_{c,1} \vee x_{c,2}\right).$$

Дизъюнкты рёбер идут по одному на каждый цвет.

$$\left(\overline{x_{a,1}} \vee \overline{x_{b,1}}\right) \wedge \left(\overline{x_{a,1}} \vee \overline{x_{c,1}}\right) \wedge \left(\overline{x_{b,1}} \vee \overline{x_{c,1}}\right) \wedge$$
$$\left(\overline{x_{a,2}} \vee \overline{x_{b,2}}\right) \wedge \left(\overline{x_{a,2}} \vee \overline{x_{c,2}}\right) \wedge \left(\overline{x_{b,2}} \vee \overline{x_{c,2}}\right).$$

Попарные запреты «не более одного цвета» опустим, ведь для противоречия они не нужны.

Вывод для треугольника

Пример

Цветов два, а вершин три, поэтому какие-то две вершины получили один цвет.

Эти две смежны, и их рёберный дизъюнкт запрещает совпадение.

Формула невыполнима — треугольник нельзя раскрасить в два цвета.

Ответ UNSAT — доказательство несуществования.

Ответ SAT — готовая раскраска, и модель формулы читается как решение задачи.

Судоку и N-ферзи

Запрет двух одинаковых цифр в строке — это дизъюнкт $\overline{x_{r,c,d}} \vee \overline{x_{r,c',d}}$ для каждой пары клеток. Столбец и блок дают такие же пары, а клетка получает ровно одну цифру.

- Судоку кодируется переменной $x_{r,c,d}$, читаемой «в клетке (r, c) стоит цифра d». $9^3 = 729$ переменных, около девяти тысяч дизъюнктов — решатель справляется за миллисекунды.
- N-ферзи кодируются переменной $x_{i,j}$, читаемой «ферзь в клетке (i, j)». При $N = 8$ — порядка семисот дизъюнктов.

Решатель не знает правил игры, ему дан плоский список дизъюнктов.

Хорошая кодировка делает структуру задачи частью самих дизъюнктов.

SAT и NP

Трудность SAT описывается в терминах классов задач.

Определение 18 (NP-полнота)

Задача **NP-полна**, если она лежит в классе **NP** и к ней полиномиально сводится любая задача из NP.

В класс **NP** входят задачи, чей ответ «да» проверяется за полиномиальное время по предъявленному сертификату.

Для SAT сертификат — выполняющий набор, который подставляют и проверяют на каждом дизъюнкте.

Проверить легко, найти — перебор.

SAT NP-полна (Кук, 1971), поэтому полиномиальный алгоритм для SAT означал бы $P = NP$.

Практика против худшего случая

NP-полнота — свойство худшего случая.

Индустриальные формулы структурны, и CDCL решает формулы с миллионами переменных.

SAT — рабочий инструмент верификации, планирования и криптоанализа.

Эквивалентность схем с миллионами вентилей — стандартный шаг потока проектирования процессоров.

Формула митера растёт линейно по числу вентилей, и её дизъюнкты повторяют структуру схемы, поэтому распространение истины не вырождается в перебор.

Вопросы по схемам

Вопросы

1. Постройте схему для $\overline{x \wedge y} \vee z$ и посчитайте её размер и глубину.
2. Запишите формулы полусумматора и полного сумматора.
3. Почему глубина каскада сумматоров ограничивает тактовую частоту?
4. Постройте G_4 из G_3 рекурсивно.
5. Переведите 0110 из кода Грея в двоичный.
6. Выразите $x \vee y$ через одни вентили NAND и посчитайте, сколько их ушло.
7. Почему минимальная ДНФ чётности не сжимается склеиваниями?
8. Что выдаст SAT-решатель для митера схем $x \vee y$ и $x \oplus y$?

Вопросы по решателям

Вопросы

1. Проследите DPLL на формуле $(x \vee y) \wedge (\overline{x} \vee y) \wedge (x \vee \overline{y})$ — где ветвление и каков ответ?
2. Как в хорновской формуле распространение фактов строит наименьшую модель Пролога?
3. Почему дизъюнкт из трёх литералов не разлагается в пару импликаций?
4. Почему выученный дизъюнкт отсекает больше, чем простой откат?
5. Проверьте по графу импликаций выполнимость $(x \vee y) \wedge (\overline{x} \vee y) \wedge \overline{y}$.

Что почитать?

Задача	Где читать
Собрать схему	m11, разделы «Логические вентили и схемы» и «Схемная сложность»
Сложить числа	m11, разделы «Сумматоры» и «Код Грея»
Проверить схему	m11, раздел «Верификация схем через SAT»
Разобрать решатели	m14, разделы «Резолюция», «От DPLL к CDCL» и «SAT и NP»